ELTE EÖTVÖS LORÁND UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

March 6, 2026

Content in this Practice I

Queue

Queue

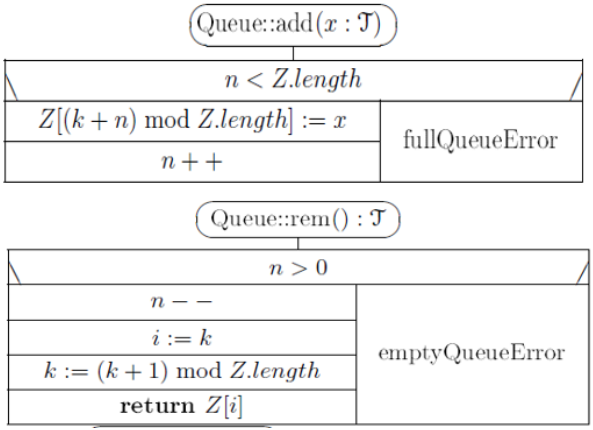
- A queue is a FIFO (First In First Out) data structure, that is, unlike a stack, the element inserted first is the one we can remove first. There are numerous implementations of the queue, e.g. with a static or dynamic array (see lecture).
- Amortized time complexity: $O(1)$

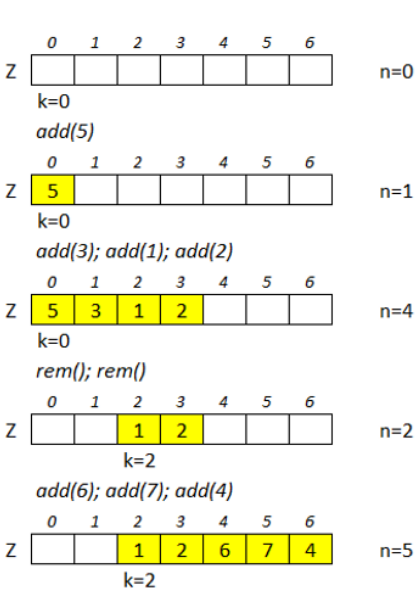
Queue
$-\ Z : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $Z.length$ is the physical
$-$ constant $m0 : \mathbb{N}_+ := 16$ // length of the queue, its default is m0.
$-\ n : \mathbb{N}$ // $n \in 0..Z.length$ is the actual length of the queue
$-\ k : \mathbb{N}$ // $k \in 0..(Z.length - 1)$: the starting position of the queue in Z
$+$ Queue($m : \mathbb{N}_+ := m0$)$\{\ Z :=$ **new** $\mathcal{T}[m]\ ;\ n := 0\ ;\ k := 0\ \}$
// create an empty queue
$+$ add($x : \mathcal{T}$) // join x to the end of the queue
$+$ rem() : $\mathcal{T}$ // remove and return the first element of the queue
$+$ first() : $\mathcal{T}$ // return the first element of the queue
$+$ length() : $\mathbb{N}$ {**return** n}
$+$ isEmpty() : $\mathbb{B}$ {**return** $n = 0$}
$+ \sim$ Queue() { **delete** Z }
$+$ setEmpty() $\{n := 0\}$ // reinitialize the queue

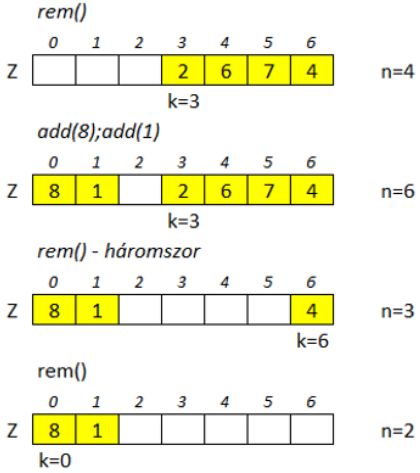
Queue

- Queue in an array:
  - The queue is cyclic, it goes “round and round”.
  - Here it is practical to index the array from zero.
  - If we know the index of the first element of the queue (k), and the number of elements in the queue (n), then we can both remove and insert in constant time.
  - First element of the queue: $Z[k]$ (if it is not empty)
  - Number of elements: n
  - First free slot: $Z[k+n \mod m]$

Queue with Array







Queue – Linked Representation

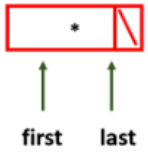
- Queue operations:
  - rem() : T
  - add(x : T)
  - first() : T
  - length() : N
  - isEmpty() : B
  - setEmpty()

- Operation cost in case of array representation: θ (1) (for all operations)
- If, for the add operation, we allow the size of the array representing the queue to increase, then its operation cost is always exactly the current length, but since this happens rarely, on average add can still be considered to have a constant number of steps.
- This must also be ensured in the case of list representation!

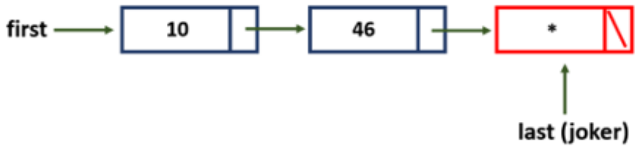
Queue – Singly Linked Representation

- Idea: A linked list with two pointers to the first and last elements.
- It contains a "joker" element at the end, which plays the role of the "next element to be inserted here".
- When we add a new element, we insert it before the joker element.
- The "last" pointer points to the joker element.
- The joker element plays the role of a reversed header, so the list is never empty.

Queue
-first, last: E1* // New pointers pointing to the first and last elements -size: N
+ Queue() + add(x: T) // Adding new element at the end + rem(): T // Removing the first element + first() : T // Query the first element + length(): N + isEmpty(): B + ~Queue() + setEmpty()



Üres sor



Queue – Singly Linked Structograms

Queue::Queue()

first := last := new E1
first->next := 0
size := 0

A konstruktor létrehozza a joker elemet.

Queue::add(x: T)

last->next := new E1
last->key := x
last := last->next
last->next := 0
size := size + 1

Queue::setEmpty()

first ≠ last	
	p := first
	first := first->next
	delete p
size:= 0	

Queue::rem(): T

size = 0	
Error	x := first->key
	s := first
	first := first->next
	delete s
	size := size - 1
	return x

Queue::first(): T

size = 0	
Error	return first->key

Queue::isEmpty(): B

return size = 0

Queue::length(): N

return size

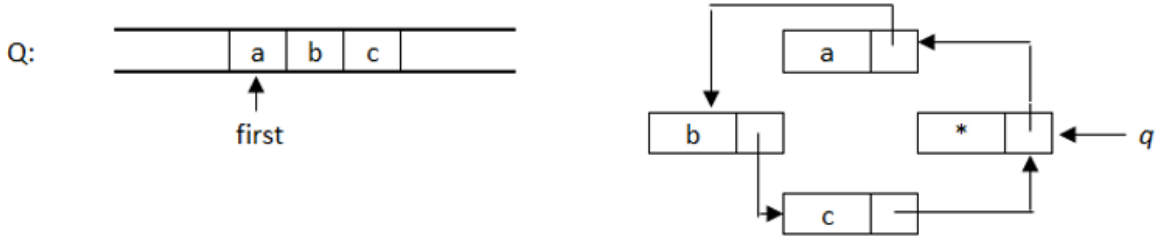
Queue::~Queue()

first ≠ 0	
	p := first
	first := first->next
	delete p

- Operation cost except for setEmpty() and the destructor : $\theta(1)$ (those are proportional to the length of the list)

Queue - Cyclic Linked Representation

- A singly linked, cyclic list represents the queue. Here too there is a joker element, located at the "end" of the queue. The joker element points to the first element.
- Only a single pointer is needed (q in the figure), which must point to the joker element.
- In the case of the rem() operation, the first element is thus immediately accessible with the help of q and can be unlinked.
- In the case of the add() operation, the value to be inserted into the queue is placed into the joker element, then we create a new joker element, link it into the chain, and make q point to this new element.



Queue - Cyclic Linked Structograms

Queue::Queue()

```
q := new E1
q->next := q
size := 0
```

Queue::add(x: T)

```
r := q->next
q->key := x
q->next := new E1
q := q->next
q->next := r
size:= size+1
```

Queue::setEmpty()

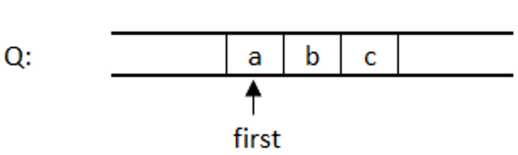
```
p := q->next
p ≠ q
    r := p
    p := p->next
    delete r
q->next := q
size:= 0
```

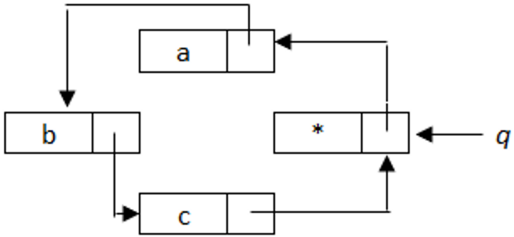
Queue::rem(): T

```
size = 0
    ERROR
else
    r := q->next->next
    x := q->next->key
    delete q->next
    q->next := r
    size:= size-1
    return x
```

Queue::~Queue()

```
p := q->next
p ≠ q
    r := p
    p := p->next
    delete r
delete q
```





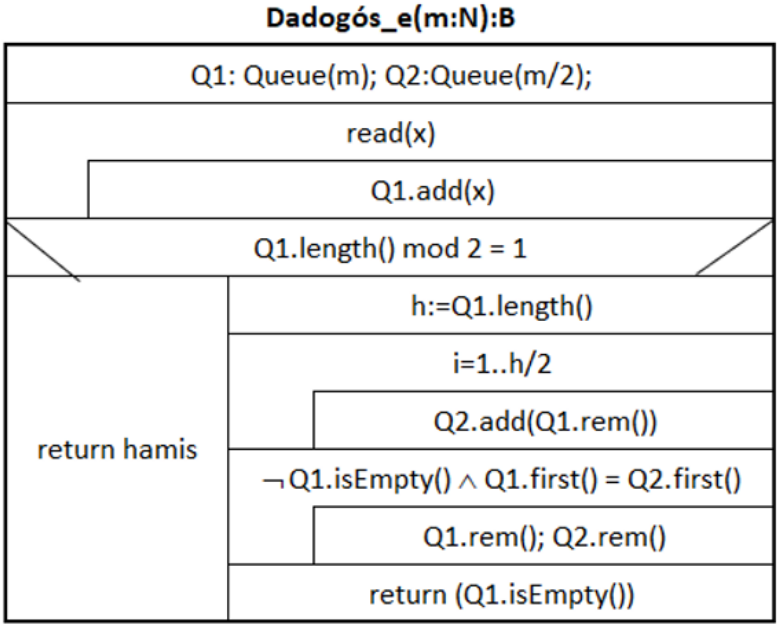
Queue Tasks

"Stuttering" Words Problem

- A text arrives from the input, and we have to decide whether the input text is “stuttering” or not.
- Example: ‘‘`aa`’’, ‘‘`appleapple`’’, `<>` (the empty text is also considered stuttering)
- Idea:
  - Read the text into a queue.
  - If the length is odd, then it is not “stuttering”.
  - If it is even, copy one half of it into another queue, then compare them.

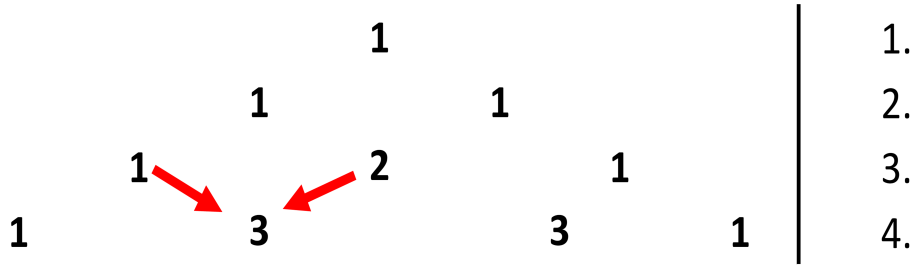
Solution: "Stuttering" Words

- Initialize the queues.
- Read the text into Q1.
- Check whether it is odd.
- Copy one half of Q1 into Q2.
- Compare the queues:
  - If they are equal, continue.
  - If empty, return with the result.

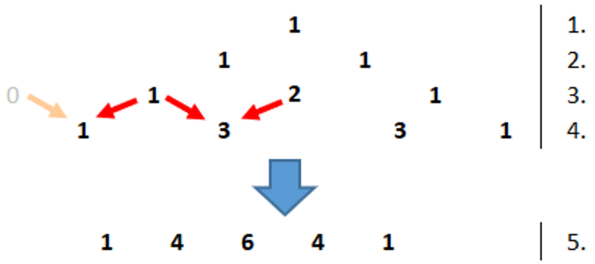


Generating Pascal's Triangle

- Using a queue, generate an arbitrary k-th row of Pascal's triangle.
- The algorithm may not use multiplication, only addition!
- (Source: https://www.mathsisfun.com/pascals-triangle.html)



Generating Pascal's Triangle



Pascal(k: N ; Q: Queue)

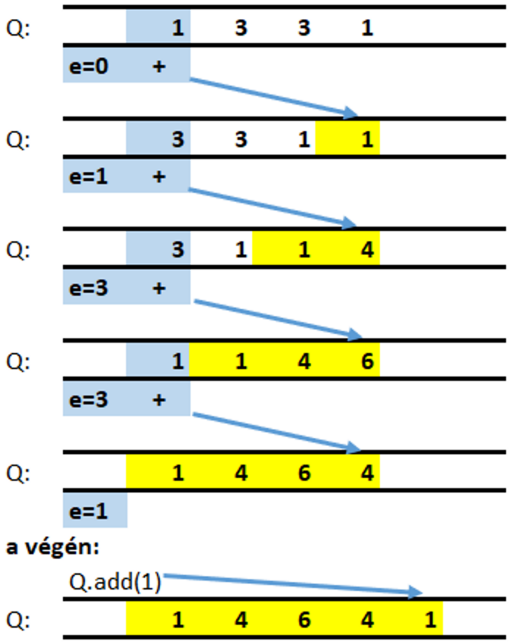
```
Q.setEmpty()
Q.add(1)
i := 2 to k
    e := 0
    j := 1 to i-1
        Q.add(e + Q.first())
        e := Q.rem()
    Q.add(1)
```

i-1-edik sorból az i-edik előállítása:

(i-1)-szer ismételjük:

Q.add(e+Q.first())

e:=Q.rem()



Generating Pascal's Triangle

```
Pascal(k: N; Q:Queue)
Q.setEmpty()
Q.add(1)
i := 2 to k
    e := 0
    j := 1 to i-1
        Q.add(e + Q.first())
        e := Q.rem()
    Q.add(1)
```

Sor, verem (nagyobb objektum) mindig referencia szerint adódik át, nem kell jelölni!

Betesszük az első sorban található 1-est, így a háromszög első sora Q-ban van.

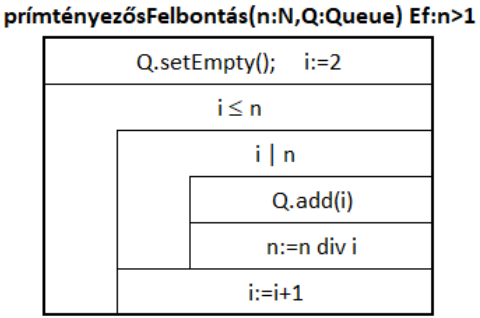
A háromszög i. sora az i-1. sor segítségével számítható ki. Amikor a ciklusba belépünk Q-ban az i-1. sor elemei vannak.

A Q sorból kiszedegetjük az elemeket, közben már állítjuk elő a háromszög i-dik sorát. Kihasználjuk, hogy az i-1. sor pontosan i-1 elemből áll, e-ben mindig az előző menetben kivett elem van, kezdetben pedig nulla.

Még egy 1-est beteszünk Q végére, ezzel a háromszög i. sora Q-ban előállt.

Merging Two Sorted Queues

- Q1 and Q2 contain the prime factorization of two natural numbers greater than 1.
- The queues are sorted in increasing order.
- In Q2, produce the prime factorization of the **least common multiple** of the two numbers.
- Preserve the contents of queue Q1!
- Example input:
  - $Q1 = \langle 2, 2, 5, 11 \rangle$
  - $Q2 = \langle 2, 3, 5, 5, 7 \rangle$
- Result:
  - $Q1 = \langle 2, 2, 5, 11 \rangle$
  - $Q2 = \langle 2, 2, 3, 5, 5, 7, 11 \rangle$



Merging Two Sorted Queues

LKKT(Q1: Queue, Q2: Queue)

	Q1.add(-1); Q2.add(-1)		*//-1 végjel berakása a sorokba*
	Q1.first() ≠ -1 ∨ Q2.first() ≠ -1		
	Q2.first() = -1 ∨ (Q1.first()≠-1 ∧ Q1.first() < Q2.first())	Q1.first()≠-1 ∧ Q2.first() ≠ -1 ∧ Q1.first() = Q2.first()	Q1.first()=-1 ∨ (Q2.first() ≠ -1 ∧ Q1.first() > Q2.first())
	Q2.add(Q1.first())	Q1.add(Q1.rem())	Q2.add(Q2.rem())
	Q1.add(Q1.rem())	Q2.add(Q2.rem())	
	Q1.rem(); Q2.rem()		*//-1 végjel eltüntetése a sorokból*

	Q1	Q2
=	2,2,5,11,-1	2,3,5,5,7,-1
<	2,5,11,-1,2	3,5,5,7,-1,2
>	5,11,-1,2,2	3,5,5,7,-1,2,2
=	5,11,-1,2,2	5,5,7,-1,2,2,3
>	11,-1,2,2,5	5,7,-1,2,2,3,5
>	11,-1,2,2,5	7,-1,2,2,3,5,5
Q2.first()=-1	11,-1,2,2,5	-1,2,2,3,5,5,7
	-1,2,2,5,11	-1,2,2,3,5,5,7,11

Ending

Thank you for your attention!